

System Identification Project

Part One: Fitting an unknown function

Professor:

Bușoniu Ioan Lucian

Supervisor:

Sântejudean Tudor Voicu

Students:

Coș Tudor-Adrian

Mastan Andrei-Mircea

Moga Anton-Ioan

Group: 30332

Group index: 2/9

Table of contents

1. Problem statement
2. Methodology
3. Key features
4. Representative plots
5. MSE Representation
6. Conclusion
7. Appendix

Problem statement

- ❖ We study the scalar output y of an unknown static nonlinear system as a function of 2 inputs x_1 and x_2 . We are given two datasets sampled on a 2D grid: an identification set $(x_1(k), x_2(k), y(k))$ and a separate validation set.
- ❖ The goal of this project is to predict y for any new pairing (x_1, x_2) by learning an approximation of the mapping $f(x_1, x_2)$ from the data.

Methodology

- ❖ In order to understand our task, we need to first know the approximator model
- ❖ Example for the 1st, 2nd and 3rd degree:

➤ $m = 1, \hat{g}(x) = [1, x_1, x_2] \cdot \Theta$

➤ $m = 2, \hat{g}(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1x_2] \cdot \Theta$

➤ $m = 3, \hat{g}(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1^3, x_2^3, x_1x_2, x_1^2x_2, x_1x_2^2] \cdot \Theta$

❖ We need to find $\begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \\ \theta_n \end{bmatrix}$ from this eq $\begin{bmatrix} y(1) \\ y(2) \\ \vdots \\ y(N) \end{bmatrix} = \begin{bmatrix} \varphi_1(1) & \varphi_2(1) & \dots & \varphi_n(1) \\ \varphi_1(2) & \varphi_2(2) & \dots & \varphi_n(2) \\ \dots & \dots & \dots & \dots \\ \varphi_1(N) & \varphi_2(N) & \dots & \varphi_n(N) \end{bmatrix} \cdot \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \\ \theta_n \end{bmatrix}$

$Y = \Phi\theta$

- ❖ We need to first generate $\phi(x)$
- ❖ We found multiple methods for generating $\phi(x)$

Methodology

- ❖ Some of our attempts at finding the formula for Phi

$p + m = 3$

$$\begin{array}{c|c|c|c|c|c|c|c}
 x_1^3 & x_2^3 & x_1^2 & x_2^2 & x_1 & x_2 & 1 & 0 \\
 \hline
 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8
 \end{array}$$

for $i = 1 : m$
 for $j = 1 : m - i$
 pt $i = 1 \Rightarrow j \rightarrow 1 : 2 \Rightarrow x_1 \cdot x_2, x_1 \cdot x_2^2$
 pt $i = 2 \Rightarrow j \rightarrow 1 \Rightarrow x_1^2 \cdot x_2$
 pt $i = 3 \Rightarrow j \rightarrow 1$

$pt. m = 3$

$i = 1 \quad j = 1 : x = x_1 - id_{vec}^1 \times x_2 - id_{vec}^1$
 $j = 0 : x = x_1^1 \times x_2^1$
 $i = 2 \quad j = 2 : x = x_1 - id_{vec}^2 \times x_2 - id_{vec}^{2-2}$
 $j = 1 : x = x_1 - id_{vec}^2 \times x_2 - id_{vec}^{2-1}$
 $j = 0 : x = x_1^2 \times x_2^2$
 $i = 3 \quad j = 3 : x = x_1^3 \times x_2^3$
 $j = 2 : x = x_1^3 \times x_2^2$
 $j = 1 : x = x_1^2 \times x_2^3$
 $j = 0 : x = x_1^3 \times x_2^3$

$phi = \begin{bmatrix} 1 & x_1 & x_2 & x_1^2 & x_2^2 & x_1^3 & x_2^3 & x_1^2 x_2 & x_1 x_2^2 & x_1^3 x_2^2 & x_1^2 x_2^3 & x_1^3 x_2^3 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

$m = 2$

for $i = 1 : m$
 for $j = 1 : m - i$
 $i = 1 \Rightarrow j \text{ del la } 1 \text{ la } 1$
 $i = 2 \Rightarrow j \text{ del la } 1$

$3 \quad m = 1 \Rightarrow 1, x_1, x_2$
 $6 \quad m = 2 \Rightarrow 1, x_1, x_2, x_1^2, x_2^2, x_1 x_2$
 $10 \quad m = 3 \Rightarrow 1, x_1, x_2, x_1^2, x_2^2, x_1^3, x_2^3, x_1 x_2, x_1^2 x_2, x_1 x_2^2$
 $15 \quad m = 4 \Rightarrow 1, x_1, x_2, x_1^2, x_2^2, x_1^3, x_2^3, x_1^4, x_2^4, x_1 x_2, x_1^2 x_2, x_1 x_2^2, x_1^3 x_2^2, x_1^2 x_2^3, x_1^4 x_2^3, x_1^3 x_2^4, x_1^4 x_2^4$

$m = 5 \Rightarrow 21$
 $m = 6 \Rightarrow 28$
 $m = 7 \Rightarrow 36$

$1 + 2 + 3 + 4 + 5 + 6 + 7$

$m = 1, counter = 1$
 $i = 0, j = 0 \Rightarrow x = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \Rightarrow phi(1)$
 $j = 1 \Rightarrow x = x_2 \Rightarrow phi(2)$
 $i = 1, j = 0 \Rightarrow x = x_1 \Rightarrow phi(3)$

$m = 2, counter = 1$
 $i = 0, j = 0 \Rightarrow x = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \Rightarrow phi(1)$
 $j = 1 \Rightarrow x = x_2 \Rightarrow phi(2)$
 $j = 2 \Rightarrow x = x_2^2 \Rightarrow phi(3)$
 $i = 1, j = 0 \Rightarrow x = x_1 \Rightarrow phi(4)$
 $j = 1 \Rightarrow x = x_1 x_2 \Rightarrow phi(5)$
 $i = 2, j = 0 \Rightarrow x = x_1^2 \Rightarrow phi(6)$

Methodology

❖ Best solution:

~~n=3~~ n=3

for i=0: n
for j=0: n-i

$1, x_1, x_2, x_1^2, x_2^2, x_1^3, x_2^3$

i=0 j=0 $x_1^0 \cdot x_2^0$ (1)
j=1 $x_1^0 \cdot x_2^1$ (2)
j=2 $x_1^0 \cdot x_2^2$ (3)
j=3 $x_1^0 \cdot x_2^3$ (4)

i=1 j=0 $x_1^1 \cdot x_2^0$ (5)
j=1 $x_1^1 \cdot x_2^1$ $x_1 x_2$
j=2 $x_1^1 \cdot x_2^2$ $x_1 x_2^2$

i=2 j=0 $x_1^2 \cdot x_2^0$ (6)
j=1 $x_1^2 \cdot x_2^1$ $x_1^2 x_2$

i=3 j=0 $x_1^3 \cdot x_2^0$ (7)

pt. n=1

i=0 j=0 $x_1^0 \cdot x_2^0$
j=1 $x_1^0 \cdot x_2^1$
i=1 j=0 $x_1^1 \cdot x_2^0$

pt. h: $\begin{pmatrix} 1 & x_2 & x_1 \\ 1 & & \\ & & \end{pmatrix}$

Methodology

In order to find theta, we use the following method:

- ❖ First, we start with: $Y = \Phi\Theta$
- ❖ Then, we left-multiply by Φ^T : $\Phi^TY = \Phi^T\Phi\Theta$
- ❖ Finally, we left-multiply by the inverse of $\Phi^T\Phi$: $\Theta = (\Phi^T\Phi)^{-1}\Phi^TY$
- ❖ We now can calculate the approximated output on the validation set, using theta: $Y_{\text{approx}} = \Phi\Theta$

Key features

❖ Finally, we settled for 3 ways of generating Phi

1. Column by column

```
283      %% functii generare big_phi
284
285      function [big_phi_matrix] = big_phi_generator(degree,input1,input2)
286
287          big_phi_matrix=[];
288
289          for i=0:degree
290              for j=0:degree-i
291                  x=(input1.^i).*(input2.^j);
292                  big_phi_matrix=[big_phi_matrix, x];
293              end
294          end
295
296      end
```

Key features

2. Column by column optimised

```
298 % a more time efficient method
299 function [big_phi_matrix] = optimised_big_phi_generator(degree,input1,input2)
300
301     size=(degree+1)*(degree+2)/2;
302     big_phi_matrix=zeros(length(input1)*length(input2),size);
303
304     contor=1;
305
306     [input1_grid, input2_grid] = meshgrid(input1, input2);
307     input1_vec=input1_grid(:);
308     input2_vec=input2_grid(:);
309
310     for i=0:degree
311         for j=0:degree-i
312             x=(input1_vec.^i).*(input2_vec.^j);
313             big_phi_matrix(:,contor)=x;
314             contor=contor+1;
315         end
316     end
317
318 end
```

Key features

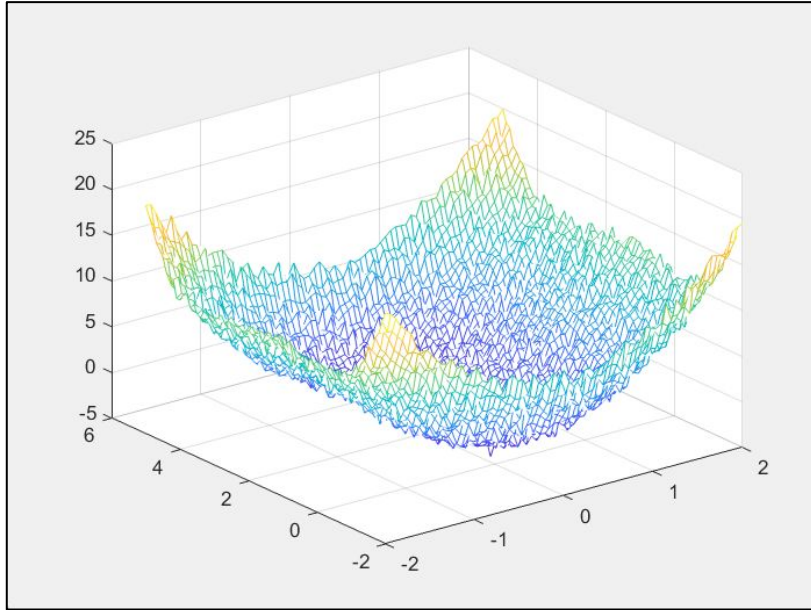
3. Element by element

```
320 % another way: element by element
321 function [big_phi_matrix] = element_wise_big_phi_generator(degree,input1,input2)
322
323     size=(degree+1)*(degree+2)/2;
324     big_phi_matrix=zeros(length(input1)*length(input2),size);
325
326     [input1_grid, input2_grid] = meshgrid(input1, input2);
327     input1_vec=input1_grid(:);
328     input2_vec=input2_grid(:);
329
330     for k=1:(length(input1)*length(input2))
331
332         index=0;
333         for i=0:degree
334             for j=0:degree-i
335                 index=index+1;
336                 term=input1_vec(k)^i*input2_vec(k)^j;
337                 big_phi_matrix(k,index)=term;
338             end
339         end
340
341     end
342
343 end
```

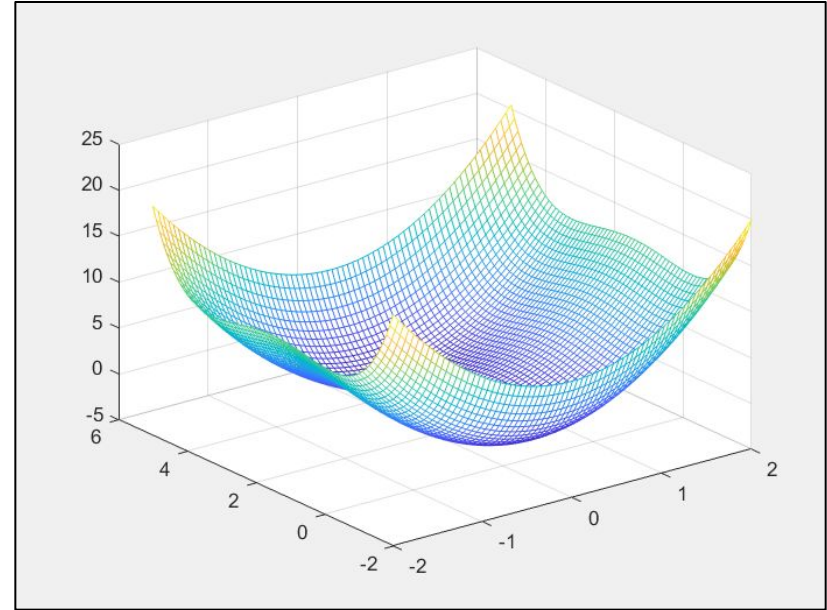
Key features

- ❖ 2 methods of obtaining the optimal degree:
 - Compute the approximation and the MSE for every degree and store all the data, then find the minimum MSE and its corresponding degree from that data
 - Find the minimum value of MSE as you compute the approximations, without storing the data
- ❖ We can use either of these methods depending on the requirements:
 - The first method for better analysis and side-by-side comparisons
 - The second method for efficiency in both speed and memory

Representative plots

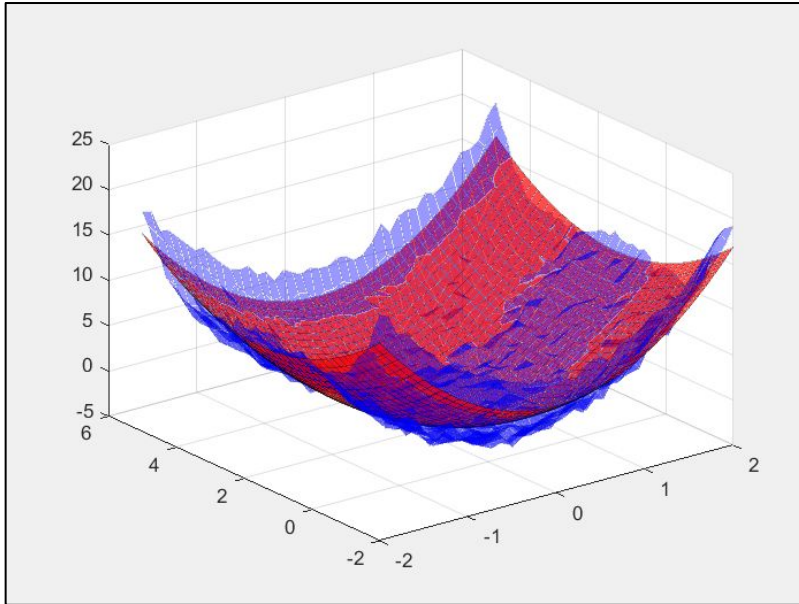


Real output for optimal degree ($n_{\min}$) - validation data

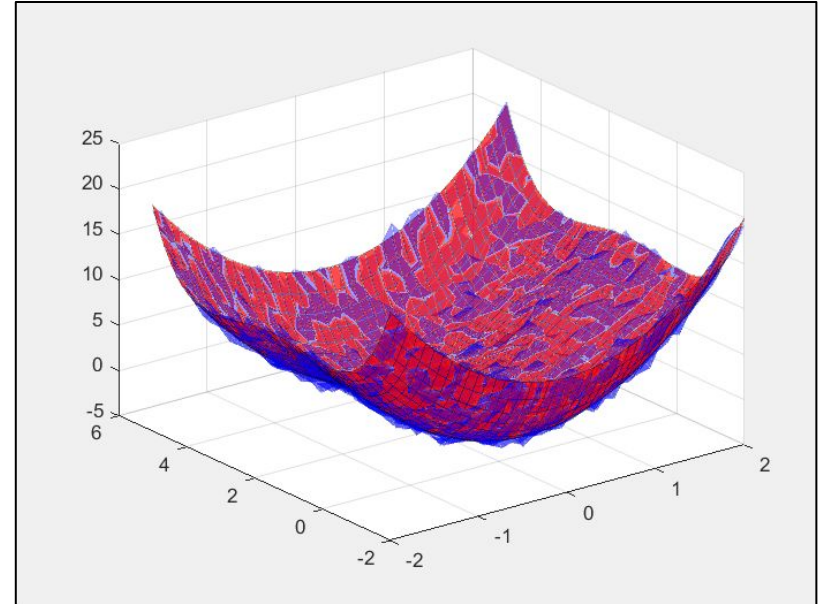


Approximated output for optimal degree ($n_{\min}$) - validation data

Representative plots

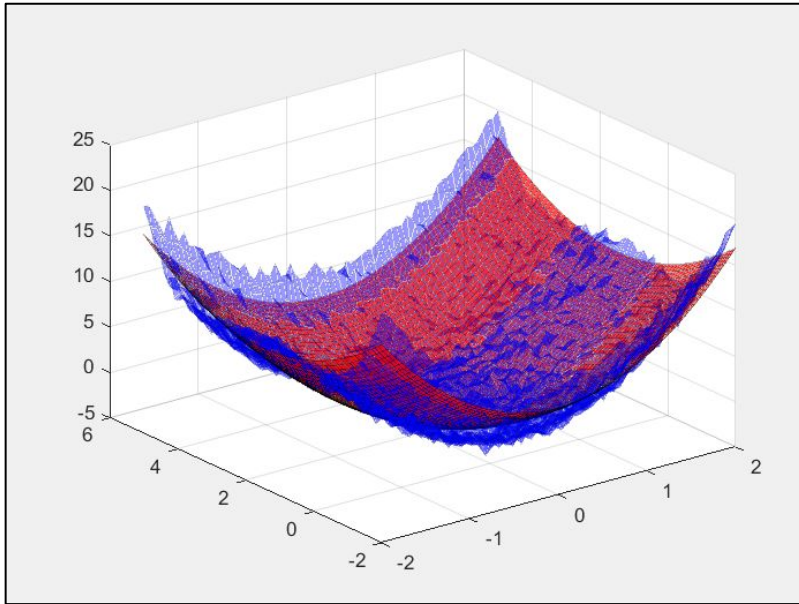


Overlap of real and approximated outputs for non-optimal degree ($n=3$) - identification data

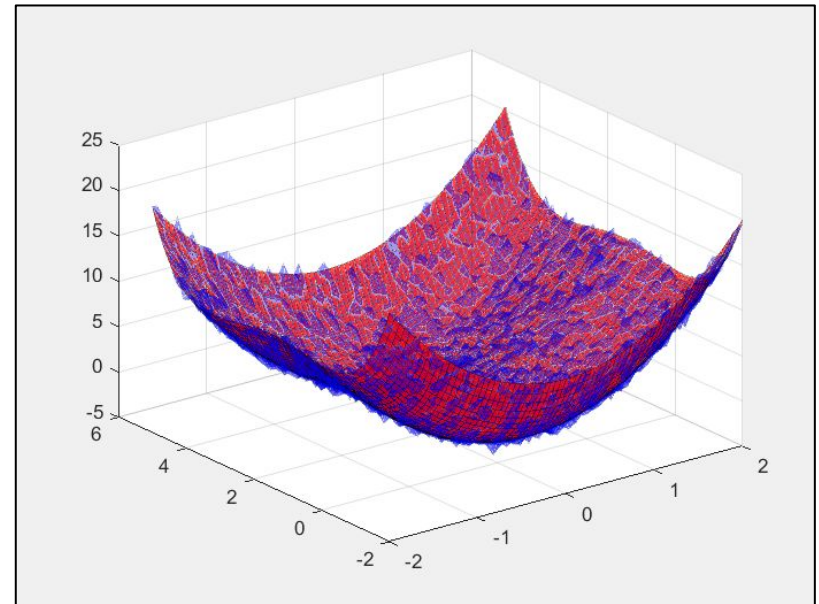


Overlap of real and approximated outputs for optimal degree ($n_{\min}$) - identification data

Representative plots

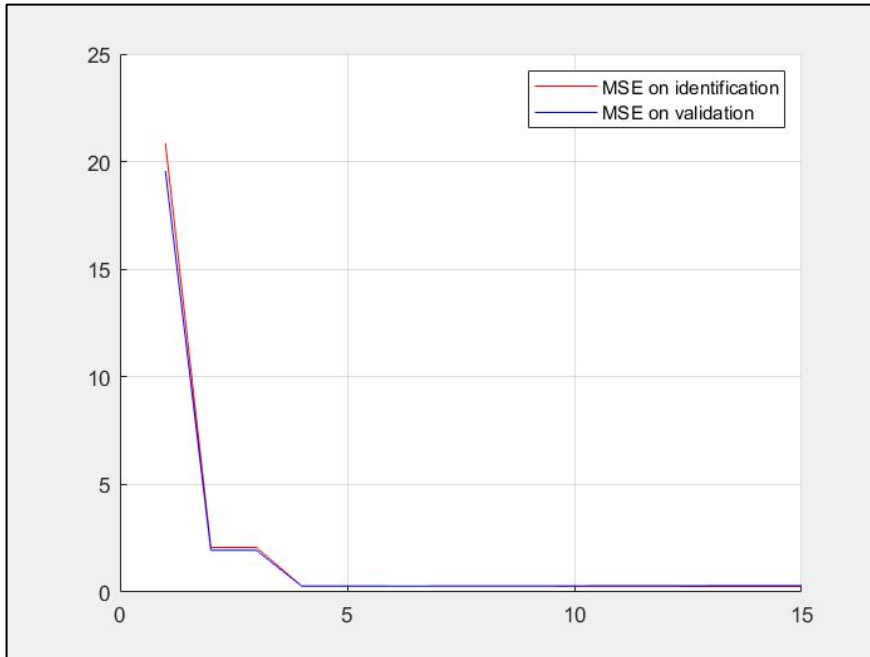


Overlap of real and approximated outputs for non-optimal degree ($n=3$) - validation data



Overlap of real and approximated outputs for optimal degree ($n_{\min}$) - validation data

MSE Representation



- ❖ We obtain the optimal degree ($n_{\min}$) by finding the minimum value of MSE in the MSE vector

```
%mse val
suma=0;
for i=1:length(Y_val_vec)
    suma=suma + (y_aprox_val(i)-Y_val_vec(i))^2;
end
MSE_vector_val(k)= 1/length(Y_val_vec)*suma;

end

%find the smallest MSE value on the validation set
[~,n_min]=min(MSE_vector_val);
```

Conclusions

- ❖ The validation curve succeeded in showing the expected values
- ❖ Degrees smaller than the optimal degree ($n_{\min}$) result in underfitting
- ❖ Degrees bigger than the optimal degree ($n_{\min}$) start to overfit
- ❖ Surface overlays confirm good match on both sets, identification and validation
- ❖ The method using preallocated/optimised Φ resulted in the same model with faster execution - useful for larger grids

Appendix

```
1 %% project part 1
2
3 % initialise variables
4 clc
5 clear
6 load('proj_fit_02');
7
8 n=3;
9
10 X1_id=id.X{1};
11 X2_id=id.X{2};
12 N1_id=length(X1_id);
13 N2_id=length(X2_id);
14
15 mesh(X1_id,X2_id,id.Y);
16
17 %copiază linia x1_id de 41 de ori, adică face o matrice de 41 pe 41
18 [X1_id_grid, X2_id_grid] = meshgrid(X1_id, X2_id);
19
20 % stack-ează coloanele una peste alta / vectorizează
21 X1_id_vec = X1_id_grid(:);
22 X2_id_vec = X2_id_grid(:);
23
24 % same here / vectorizează Y_id
25 Y_id=[];
26 for i=1:N1_id
27     Y_id=[Y_id ; id.Y(:,i)];
28 end
```

```

30 % genereaza Phi mare coloana cu coloana
31 big_phi_id=[];
32
33 for i=0:n
34     for j=0:n-i
35         x=(X1_id_vec.^i).*(X2_id_vec.^j);
36         big_phi_id=[big_phi_id, x];
37     end
38 end
39
40 % afla theta
41 Theta=big_phi_id\Y_id;
42
43 % aflam y_aprox_id
44 y_aprox_id=big_phi_id*Theta;
45
46 %comparing the shapes
47 y_aprox_id_matrix=reshape(y_aprox_id,N1_id,N2_id);%readuce la forma de matrice
48
49 figure(2);
50 mesh(X1_id,X2_id,y_aprox_id_matrix);
51
52 figure(3)
53 surf(X1_id, X2_id, id.Y,'FaceColor', 'blue', 'EdgeAlpha', 0.1, 'FaceAlpha', 0.4);
54 hold on;
55 surf(X1_id, X2_id, y_aprox_id_matrix,'FaceColor', 'red', 'EdgeAlpha', 0.3, 'FaceAlpha', 0.7);
56

```

```

57 %calc mse pe identificare
58 suma=0;
59 for i=1:length(Y_id)
60     suma=suma + (y_aprox_id(i)-Y_id(i))^2;
61 end
62 MSE= 1/length(Y_id)*suma;
63 MSE
64
65 %% for validation set
66
67 %initializare date
68 X1_val=val.X{1};
69 X2_val=val.X{2};
70 N1_val=length(X1_val);
71 N2_val=length(X2_val);
72
73 mesh(X1_val,X2_val,val.Y);
74
75 [X1_val_grid, X2_val_grid] = meshgrid(X1_val, X2_val);
76
77
78 X1_val_vec = X1_val_grid(:);
79 X2_val_vec = X2_val_grid(:);
80
81 Y_val_vec=val.Y(:);
82

```

```

83 %calc phi mare pt validare
84 big_phi_val=big_phi_generator(n,X1_val_vec,X2_val_vec);
85
86 y_aprox_val=big_phi_val*Theta;
87
88 %comparing the shapes
89 y_aprox_val_matrix=reshape(y_aprox_val,N1_val,N2_val);
90 figure(2);
91
92 mesh(X1_val,X2_val,y_aprox_val_matrix);
93
94 figure(3)
95 surf(X1_val, X2_val, val.Y,'FaceColor', 'blue', 'EdgeAlpha', 0.1, 'FaceAlpha', 0.4);
96 hold on;
97
98 surf(X1_val, X2_val, y_aprox_val_matrix,'FaceColor', 'red', 'EdgeAlpha', 0.3, 'FaceAlpha', 0.7);
99
100 %calc mse validare
101 suma=0;
102 for i=1:length(Y_val_vec)
103     suma=suma + (y_aprox_val(i)-Y_val_vec(i))^2;
104 end
105 MSE_val= 1/length(Y_val_vec)*suma;
106 MSE_val
107

```

```

108 %% aflam n optim
109
110 % am ales gradul maxim 15 pentru a scapa de warning: rank defficiency
111
112 %am mai gasit si o varianta mai rapida de a forma big_phi_id cu suma gauss
113
114 %vectors to store MSE values
115 MSE_vector_val=zeros(15,1);
116 MSE_vector_id=zeros(15,1);
117
118 %to store the approximated outputs
119 date_id= zeros(N1_id*N2_id,15);
120 date_val= zeros(N1_val*N2_val,15);
121
122 % Y vectorizat in alta metoda
123 Y_id_vec=id.Y(:);
124 Y_val_vec=val.Y(:);
125
126 for k=1:15
127
128     n=k;
129
130     % obtaining the model for each n
131     big_phi_id=optimised_big_phi_generator(n,X1_id,X2_id);
132
133     Theta=big_phi_id\Y_id_vec;

```

```

135     y_aprox_id=big_phi_id*Theta;
136     date_id(:,k)=y_aprox_id;
137
138     %mse pt id
139     suma=0;
140     for i=1:length(Y_id_vec)
141         suma=suma + (y_aprox_id(i)-Y_id_vec(i))^2;
142     end
143     MSE_vector_id(k) = 1/length(Y_id)*suma;
144
145     %vaididare
146
147     big_phi_val=element_wise_big_phi_generator(n,X1_val,X2_val);
148
149     y_aprox_val=big_phi_val*Theta;
150     date_val(:,k)=y_aprox_val;
151
152     %mse val
153     suma=0;
154     for i=1:length(Y_val_vec)
155         suma=suma + (y_aprox_val(i)-Y_val_vec(i))^2;
156     end
157     MSE_vector_val(k)= 1/length(Y_val_vec)*suma;
158
159 end

```

161	%find the smallest MSE value on the validation set	
162	[~,n_min]=min(MSE_vector_val);	
163	n_min	
164		
165	%% plotting the best approx on the id set	
166	n=1:15;	
167	figure(1)	
168	hold on;	
169	plot(n,MSE_vector_id,'r');grid;	
170	plot(n,MSE_vector_val,'b');	
171	legend("MSE on identification","MSE on validation")	
172		
173	figure(2)	
174	mesh(X1_id,X2_id,id.Y);	
175		
176	y_aprox_id_matrix=reshape(date_id(:,n_min),N1_id,N2_id);	
177	figure(3);	
178		
179	mesh(X1_id,X2_id,y_aprox_id_matrix);	
180		
181	figure(4)	
182	surf(X1_id,X2_id,id.Y,'FaceColor','blue','EdgeAlpha',0.1,'FaceAlpha',0.4);	
183	hold	
184	surf(X1_id,X2_id,y_aprox_id_matrix,'FaceColor','red','EdgeAlpha',0.3,'FaceAlpha',0.7);	
185		

```
186 %% plotting the best approximation for validation
187 n=1:15;
188 figure(1)
189 hold on;
190 plot(n,MSE_vector_id,'r');grid;
191 plot(n,MSE_vector_val,'b');
192 legend("MSE on identification","MSE on validation")
193
194 figure(2)
195 mesh(X1_val,X2_val,val.Y);
196
197 y_aprox_val_matrix=reshape(date_val(:,n_min),N1_val,N2_val);
198 figure(3);
199
200 mesh(X1_val,X2_val,y_aprox_val_matrix);
201
202 figure(4)
203 surf(X1_val,X2_val,val.Y,'FaceColor','blue','EdgeAlpha',0.1,'FaceAlpha',0.4);
204 hold
205 surf(X1_val,X2_val,y_aprox_val_matrix,'FaceColor','red','EdgeAlpha',0.3,'FaceAlpha',0.7);
```

```

208 %% calculare n optim varianta fara vectori de date
209
210 MSE_vector_val=zeros(15,1);
211 MSE_vector_id=zeros(15,1);
212 n_optim=1;
213 for k=1:15
214
215     n=k;
216
217     big_phi_id=[];
218
219     for i=0:n
220         for j=0:n-i
221             x=(X1_id_vec.^i).*(X2_id_vec.^j);
222             big_phi_id=[big_phi_id, x];
223         end
224     end
225
226     Theta=big_phi_id\Y_id_vec;
227
228     y_aprox_id=big_phi_id*Theta;
229
230     suma=0;
231     for i=1:length(Y_id_vec)
232         suma=suma + (y_aprox_id(i)-Y_id_vec(i))^2;
233     end
234     MSE_vector_id(k) = 1/length(Y_id)*suma;

```

```

236 %validare
237 big_phi_val=[];
238
239 for i=0:n
240     for j=0:n-i
241         x=(X1_val_vec.^i).*(X2_val_vec.^j);
242         big_phi_val=[big_phi_val, x];
243     end
244 end
245
246 y_aprox_val=big_phi_val*Theta;
247
248 suma=0;
249 for i=1:length(Y_val_vec)
250     suma=suma + (y_aprox_val(i)-Y_val_vec(i))^2;
251 end
252 MSE_vector_val(k)= 1/length(Y_val_vec)*suma;
253
254 if k==1
255     MSE_n_optim=MSE_vector_val(k);
256     n_optim=k;
257     y_aprox_val_n_optim=y_aprox_val;
258 end
259
260 if MSE_vector_val(k)<MSE_n_optim
261     MSE_n_optim=MSE_vector_val(k);
262     n_optim=k;
263     y_aprox_val_n_optim=y_aprox_val;
264 end
265 end

```

```
266     n_optim
267     MSE_n_optim
268
269
270     figure(1)
271     hold on;
272     plot(MSE_vector_id,'r');grid;
273     plot(MSE_vector_val,'b');
274     legend("MSE on identification","MSE on validation")
275
276     figure(2)
277     surf(X1_val, X2_val, val.Y,'FaceColor', 'blue', 'EdgeAlpha', 0.1, 'FaceAlpha', 0.4);
278     hold on;
279     y_aprox_val_n_optim=reshape(y_aprox_val_n_optim,N1_val,N2_val);
280     surf(X1_val, X2_val, y_aprox_val_n_optim,'FaceColor', 'red', 'EdgeAlpha', 0.3, 'FaceAlpha', 0.7);
281
282
```

```
283 %% functii generare big_phi
284
285 function [big_phi_matrix] = big_phi_generator(degree,input1,input2)
286
287     big_phi_matrix=[];
288
289     for i=0:degree
290         for j=0:degree-i
291             x=(input1.^i).*(input2.^j);
292             big_phi_matrix=[big_phi_matrix, x];
293         end
294     end
295
296 end
```

```

298 % a more time efficient method
299 function [big_phi_matrix] = optimised_big_phi_generator(degree,input1,input2)
300
301     size=(degree+1)*(degree+2)/2;
302     big_phi_matrix=zeros(length(input1)*length(input2),size);
303
304     contor=1;
305
306     [input1_grid, input2_grid] = meshgrid(input1, input2);
307     input1_vec=input1_grid(:);
308     input2_vec=input2_grid(:);
309
310     for i=0:degree
311         for j=0:degree-i
312             x=(input1_vec.^i).*(input2_vec.^j);
313             big_phi_matrix(:,contor)=x;
314             contor=contor+1;
315         end
316     end
317
318 end

```

```

320 % another way: element by element
321 function [big_phi_matrix] = element_wise_big_phi_generator(degree,input1,input2)
322
323     size=(degree+1)*(degree+2)/2;
324     big_phi_matrix=zeros(length(input1)*length(input2),size);
325
326     [input1_grid, input2_grid] = meshgrid(input1, input2);
327     input1_vec=input1_grid(:);
328     input2_vec=input2_grid(:);
329
330     for k=1:(length(input1)*length(input2))
331
332         index=0;
333         for i=0:degree
334             for j=0:degree-i
335                 index=index+1;
336                 term=input1_vec(k)^i*input2_vec(k)^j;
337                 big_phi_matrix(k,index)=term;
338             end
339         end
340
341     end
342
343 end

```